Arcron — The Working Guide

A permissionless keeper network for Algorand: from your first upkeep to running the network

Compiled from the Arcron project documentation (CorvidLabs)

August 2026 · TestNet app 769891898 · alpha, unaudited

Contents

Preface

What this book is

Arcron is a **permissionless keeper network for Algorand**. A smart contract cannot wake itself up: something off-chain has to call it. Arcron lets anyone register a scheduled contract call with escrowed ALGO, which *any* keeper can then execute for the fee. No allowlist, no stake, no token, no owner.

This book is a single, ordered path through everything the Arcron project documents — the concept, the console, integrating your own contract, running a keeper, the security model, the economics, and the full technical reference. It was compiled from the project's own documentation and source. Where the project states a number, this book keeps that number and says where it comes from, so you can check it.

It is written to be read front to back by someone new, and dipped into later by someone building. Each chapter opens by telling you who it is for and what you will be able to do at the end.

The state of the thing, honestly

The Arcron project is unusually candid about its own maturity, and this book inherits that. Four facts frame everything else:

- **It is alpha, unaudited, and TestNet only.** The one deployment to use is app `769891898`; every other app id is superseded or a look-alike (see the quarantine, Chapter 9). Do not put MainNet value into any of them. "Alpha" also means this app id may be replaced for any reason — cancelling is how you leave, and it refunds your escrow and box deposit.
- **The contract is upgradeable until its creator "freezes" it**, and it is not frozen today. Until then, the creator can replace its programs and reach every escrow. This is disclosed on-chain and checkable; see Chapter 9.
- **Every keeper running is the project's own.** "Permissionless" is true architecturally and, for now, false empirically.
- **Nobody outside the project has registered an upkeep yet.** The project has staked its thesis on that changing. See Chapter 11.

None of these is hidden, and none is a reason not to learn the system. They are the reason to learn it *before* relying on it.

How to read this book

The book has five parts and a reference section.

Part	You are	Read it to
I — Understanding Arcron	anyone	get the idea, the roles, and the mechanism
II — Using Arcron (creator)	scheduling a call	register an upkeep and hook up your own contract
III — Running a keeper	earning fees	stand up a keeper and operate it
IV — Trust, security, economics	deciding whether to rely on it	judge the safety and the cost honestly
V — Reference	building against it	look up signatures, encodings, commands

Two short paths through it:

- **You want the idea.** Read Chapters 1 and 2 (about ten minutes).
- **You want to *use* it.** Read the four facts above, then Chapter 2, then Chapter 4 — that is enough to register your first upkeep. Come back for the rest.

If you came to *break* it or to judge it, read Part I then Part IV.

Conventions

- **Money.** Amounts on-chain are in **microALGO (µALGO)**; 1 ALGO = 1,000,000 µALGO. The contract's floor fee is 4,000 µALGO = 0.004 ALGO.
- **Time.** Arcron schedules in **rounds**, not seconds. A round is roughly 2.66–2.8 seconds. "Rounds are not a clock" is a running theme; Chapter 6 explains why it matters.
- **Callout boxes** (indented quotes) flag the things that cost people an hour, or that are safety-critical.
- **Commands** are shown exactly as you would run them. Anything that writes to a chain is called out as such.

A note on the numbers

The project's own documentation warns: *"Do not trust this page's numbers. Several of them were wrong last week and were corrected by a review. Recompute anything you intend to rely on."* This book carries that spirit forward. The figures here are the project's stated measurements as of August 2026; treat them as checkable claims, not gospel, and re-derive anything load-bearing against the live chain. This edition was itself fact-checked against the repository and corrected; a couple of figures the source docs disagree with themselves on are flagged where they appear.

Part I — Understanding Arcron

Chapter 1 — The problem: a contract cannot wake itself up

For everyone. By the end you will understand why "call this later" is a genuine gap on Algorand, and why previous attempts to fill it are worth studying.

The one thing smart contracts cannot do

A smart contract is reactive. It runs only when a transaction calls it. It has no thread, no timer, no `cron`. If you want a contract method to run at 09:00, or every hour, or once a prize window closes, *something outside the chain* must send the transaction that calls it.

Today, on Algorand, that "something" is a bot you write, host, key, and monitor. You spin up a small server, give it a hot wallet, and have it poll the clock and fire the call. It works. It is also a server, a key, and an on-call rotation for what should be one line: *"call this later."*

Algorand has no scheduling primitive

This is the whole claim, and it is smaller than "agents need this" and larger than "we built a bot":

> Every serious chain should let you say *"call this later"* without running a server. Algorand has no way to do that.

There is **no ARC** — no Algorand Request for Comments — covering scheduled execution, automation, or keeper incentives, and there never has been, not even as a submitted draft. The ecosystem hand-rolls it. The Foundation's own staking contracts (Réti) carry a method marked `// Note: ANYONE can call this` with no reward attached — permissionless as a liveness fallback, with the economic half missing. The documented answer to recurring work is still "run your own watcher on a cron."

The predecessors, and what they teach

Arcron is not the first to notice this gap, and the failures of those who tried are the most useful thing to study — because every one of them failed at the same seam: adoption, not engineering. (Chapter 11 returns to this in depth; here is the short version.)

- **AlgoRhythm** (January 2026) — a draft scheduler pushed by the CTO of AlgoNode, one of the people best placed in the ecosystem to specify this. Two commits, same day, never touched again, ending at a TODO whose entries include *"fee structure (anti-spam + incentives)."* The

hard part it stopped at was the economics. **Lesson: the incentives are the unsolved problem, not the plumbing.**

- **BiatecCron** — a keeper network that *did* ship on Algorand (MainNet app `1765620242`) and ran **three tasks in two years, all its author's own. Lesson: shipping is not adoption.**
- **Keep3r** (Ethereum) — a mature keeper network where, at one snapshot, **30 of 42 registered jobs had never been worked** and active keepers peaked at six. **Lesson: a job nobody is paid enough to run is a job that does not run.**

Hold that common thread — not "can you build a keeper network" (yes), but "can you make the incentives such that strangers actually run it, and actually use it." It returns in Chapters 10 and 11.

Why not just use AWS Lambda?

You can, and for many people you should — the project says so plainly. AWS Lambda plus EventBridge Scheduler, at the volume one schedule needs, sits deep inside a perpetual free tier, does not drift, and does not expire. If you are already on AWS, it "defeats this comparison outright." GitHub Actions cron is free too (in a private repo). So Arcron is *not* competing on raw cost against the free options.

What a self-hosted scheduler — cloud or cron — cannot give you is the one thing Part IV builds toward: a schedule that **outlives the process that created it**, needs **no hot key of yours**, and needs **no operational attention**. Whether that is worth anything is the open question the project has staked itself on. But you cannot judge it until you understand the mechanism, which is the rest of Part I.

Chapter 2 — What Arcron is

For everyone. By the end you will know the three roles, the one-minute mental model, and the honest version of the cost argument.

The shared version of "run your own bot"

Arcron is the shared version of the watcher-on-a-cron. Instead of everyone running their own server to call their own contracts, anyone can register a scheduled call once, with money attached, and any keeper can execute it for the fee. One process can service the whole registry, so the network needs a handful of keepers, not a crowd (Chapter 7 does that arithmetic).

There is **no owner and no protocol rake.** The contract holds escrow for other people and pays it out to whoever does the work. Nobody takes a cut.

The three roles

Everything in Arcron is one of three parties. Keep them straight and the rest follows.

Role	Does	Cannot
Creator	registers an upkeep, funds it, cancels their own	touch anyone else's upkeep, change a registered call, or stop a keeper executing
Keeper	executes any due, funded upkeep and collects its fee	choose *what* is called, alter a schedule, or take more than the fee the box says
Target app	does anything it likes with its own state, inside the call	reach the keeper's funds, re-enter Arcron, or change the upkeep that called it

The single guarantee the whole design rests on: **a keeper decides *when* your call happens, never *what* it says.** The call and its arguments are fixed by the creator at registration. This is what makes keepers trustless — and, as Chapter 11 shows, it is also why a keeper cannot inject fresh data (Arcron is a clock, not an oracle).

An upkeep, in one sentence

An **upkeep** is a standing instruction: *"call this app with this exact data every N rounds, paying R µALGO per execution, from this escrow."* You register it once. It runs until the escrow is empty or you cancel it. Cancelling returns everything unspent, plus the storage deposit.

The idea in one minute

1. A creator **registers** an upkeep and escrows ALGO into the contract.
2. Rounds pass. When the upkeep is **due**, any keeper may call `execute`.
3. `execute` performs the registered inner call to the target app, then pays the keeper its fee from the escrow — atomically, in one transaction, so a fee is only ever paid alongside a real execution.
4. The schedule advances. Repeat until the escrow runs low or the creator cancels.

That is the entire product. The depth is all in the corners: what happens after an outage, how the fee behaves when an upkeep is neglected, what a keeper can and cannot reach, and who can change the rules. Those corners are the rest of this book.

The honest cost case

The project is careful here, having corrected its own numbers more than once, so this book is too. One hourly schedule, run through Arcron **at the 4,000 µALGO floor fee**, costs roughly **3 ALGO a month (~$0.28 at the ALGO price used)**. Against paid hosts you would otherwise run a bot on:

One hourly schedule, per month	Cost	What you give up
Arcron (at the 4,000 µALGO floor)	~3.03 ALGO ~ $0.28	not nothing — see below
fly.io shared-cpu-1x	~$2.02	you write, host, key, and monitor the bot
Hetzner CX22	~$4.10	as above
AWS Lambda + EventBridge	$0.00	as above, but genuinely free at this volume
GitHub Actions cron (private repo)	$0.00	delayed under load, runs may be dropped

So Arcron is **several times cheaper than the cheapest *paid* host**, and **not cheaper at all than the free options.** Both halves are true and the project says both. What you actually pay Arcron for is not the cheapest possible cost; it is the absence of a hot key, a host, and an on-call rotation, and a schedule that survives you.

> **A caveat this book flags, since it invites you to check.** The ~3.03 ALGO figure is the *floor*, not what the console suggests. The console suggests a **0.010 ALGO** fee (about $0.65/month), because at the floor a keeper barely breaks even and will not reliably run your upkeep — Chapter 10 has the full reasoning. And the cost argument

mixes a drift-adjusted ~3.03 ALGO (a nominal "hour" actually fires every ~57 minutes) with a nominal 2.88 ALGO, and quotes a "cheaper" multiple that does not quite reproduce from its own table. The direction is right; the third digit is soft. Recompute against the fee you actually set.

What it does *not* do

Arcron is **the clock, not the eyes.** It schedules on-chain calls; it cannot observe the world, cannot fetch off-chain data, and cannot let a keeper supply values. If your automation needs *data*, you pair Arcron with an oracle: the oracle holds the data, Arcron guarantees the timing. Chapter 5 (Lesson 7) and Chapter 11 draw that boundary precisely, because it is exactly where the design says "no" on purpose.

Chapter 3 — How it works, end to end

For everyone who will build against it. By the end you will be able to draw the data flow and name where the money and the state live.

The architecture on one page

```
creator                  keeper app (769891898)              target app
  |  register + escrow ALGO  |                                    |
  |------------------------->|  box "u"+id : the Upkeep struct    |
  |                          |                                    |
keeper bot                 |                                    |
  |  execute(upkeep_id) ---->|  inner app call ----------------->|   (your hook
   ↪  runs)
  |                          |  inner payment (fee) ---> keeper   |
```

Three things to take from the diagram:

- **One box per upkeep.** Each upkeep lives in its own box, named `b"u"` followed by the upkeep's id. The registry is fully on-chain and readable with free algod box queries — **no indexer is required.** A keeper is a loop over boxes.
- **`execute` is atomic.** The target call and the keeper's payment are inner transactions of the *same* `execute` call. A fee is paid only alongside a real execution; there is no path that pays a keeper for work not done.
- **The contract is passive.** "Arcron is running" always means *somebody's bot is running.* There is no on-chain timer anywhere in this picture.

The lifecycle of an upkeep

An upkeep moves through a small number of states. Learning them now makes every later chapter easier.

1. **Registered.** The creator sends a transaction group that funds the box's storage deposit and the escrow, and names the target, the call data, the interval, the fee, and the missed-run policy. The contract stores the `Upkeep` struct in a fresh box and returns its id.
2. **Due / not due.** The upkeep has a `next_execution_round`. Before that round, `execute` is refused ("Not due"). At or after it, any keeper may execute.

3. **Executed.** A keeper calls `execute`. The contract advances the schedule, deducts the fee from the escrow, performs the inner call to the target, and pays the keeper. It records the round it actually ran in.
4. **Dormant (starved).** When the escrow falls below one fee, no keeper can execute it. It is not broken — it resumes the instant anyone tops it up.
5. **Cancelled.** The creator calls `cancel`. The box is deleted; its storage deposit is released and refunded along with the remaining escrow and any unspent asset bonus. The upkeep's id is never reused.

What is stored, conceptually

Each upkeep box holds one `Upkeep` struct. You do not need its byte layout yet (Appendix B has it), but you should know the fields it carries, because they are the vocabulary of the rest of the book:

Field	What it means
`creator`	who may cancel it, and who refunds go to
`target_app`, `call_args`	*what* is called, fixed forever at registration
`interval_rounds`	the cadence, in rounds
`next_execution_round`	when it is next due
`fee_per_execution`	the base fee a keeper is paid
`balance`	the ALGO escrow remaining
`policy`	`CATCH_UP` or `SKIP_AHEAD` — what happens after a missed run
`fee_cap`	the most one run may ever pay (escalation ceiling); 0 = off
`last_serviced_round`	the round it last ran; escalation is measured from here
`fee_asset`, `asset_fee`, `asset_balance`	an optional bonus paid in an ASA, *on top of* the ALGO fee
`times_executed`	a running count

Two of these fields carry more subtlety than they look — `policy` and `fee_cap` — and getting them wrong is the difference between an upkeep that survives an outage and one that burns its whole escrow. Chapter 6 is devoted to them.

The money, in and out

It helps to see, once, every way ALGO enters and leaves the contract:

- **In:** the storage deposit (box minimum balance) and the escrow, both at registration; later top-ups; and any ASA bonus escrow.
- **Out:** a keeper's fee on each execution; a refund to the creator on cancel (remaining escrow **plus** the released storage deposit); a forfeited bonus stays put until claimed or cancelled.

Every ALGO that leaves does so as **one of exactly two things**: a keeper fee or a creator refund. There is no third recipient, no owner withdrawal, and no rake. That invariant — checkable by

reading every payment the contract can emit — is the backbone of the security story in Chapter 9.

Part II — Using Arcron as a creator

Chapter 4 — Your first upkeep, from the console

For anyone with a wallet and ten minutes. By the end you will have registered a real upkeep on TestNet and watched a keeper run it — and know how to get every microALGO back.

Everything in this chapter reads or writes **TestNet only**. The worst outcome is losing a fraction of a TestNet ALGO, and even that is refundable: cancelling an upkeep returns its remaining escrow **and** its box storage deposit in full.

Why this walkthrough matters

Every *read* path in the console has been driven against live TestNet. Far fewer *write* paths have been exercised by a real wallet — `register`, `execute`, `cancel`, and `top_up` each need a signature, and no automated test can produce one. Doing this by hand is genuinely the cheapest bug-finding available. (A bug where every disabled button rendered at a 1.02:1 contrast ratio — literally invisible — survived four agent reviews, an axe-core pass at zero violations, and 91 unit tests, and was found by a human in about ninety seconds. None of those checks looks at rendered pixels.)

Before you start

- **Use only app `769891898`.** The console's canonical address is a *security property*: anyone can deploy a look-alike contract with the same form. Any other app id is superseded or hostile — the console quarantines it (Chapter 9), and you should too.
- **This is alpha and unaudited.** The deployment is **not frozen**, which means its creator can still replace its programs and reach every escrow (Chapter 9). The app id is alpha and may be replaced. Escrow only what you are willing to have on a throwaway TestNet contract.
- **Switch your wallet (Pera) to TestNet.** Settings -> Developer Settings -> Node Settings -> TestNet. Skip this and Pera hands the console a MainNet address with no TestNet balance, and the Register button stays disabled with no visible explanation.
- **Get about 0.2 TestNet ALGO.** Fund your address at https://bank.testnet.algorand.network/ or the Lora dispenser (https://lora.algokit.io/testnet/). Most of it comes back when you cancel.

Open the **canonical hosted console** and let its own URL pin the network and app:

https://corvidlabs.xyz/arcron/console/?network=testnet&app=769891898

> Check that address bar before you connect a wallet. If you would rather run the console locally instead, that is the second path: `cd web && bun install && bun run ng serve`, then open `http://localhost:4200/register?network=testnet&app=769891898`.

The numbers you will need

Keeper app	`769891898` (alpha-3)
Target app (pulse)	`769891902`
Method signature	`tick()uint64`
Selector it produces	`0x4d4d5f0b`
Box deposit	0.0621 ALGO, **refunded in full on cancel**
Minimum fee per run	0.004 ALGO (the floor; the console suggests 0.010)

`pulse` is a heartbeat counter that exists to be called. It has no state worth protecting and cannot fail, which is what makes it the right first target.

Lesson 1 — Test the call before connecting anything

Fill in **TARGET APP ID** `769891902` and **METHOD SIGNATURE** `tick()uint64`. The selector `0x4d4d5f0b` should appear as you type. Press **Test the call**.

This needs no wallet and costs nothing. It simulates the inner call Arcron will make, with the sender set to the keeper application's own account (which has no private key for anyone to hold). Checking the call *before* exposing a wallet to the page is the right order.

Expect a **graded** result, never a flat pass. A "reference" here is an extra account, asset, or app that the inner call must name; Arcron already spends two of the eight available reference slots (the upkeep box and your target), so **six** are left for a keeper to fill. For `pulse.tick()` the grade should read `RESOURCES: NONE` — the call reached for nothing a keeper must name. A `servable` grade means it needs up to six references and a keeper can discover them (Chapter 5, Lesson 6, explains how). It will also tell you what it *cannot* know — whether a keeper will turn up, and whether the call's needs will grow later.

> **If it grades anything other than `NONE` or `servable`, stop.** A target that needs more than six references is permanently unexecutable once you have escrowed. The grades exist precisely to catch that before your money is in.

Lesson 2 — Fill in the rest

Field	Value	Why
INTERVAL (ROUNDS)	`215`	about 10 minutes at ~2.66 s/round

Field	Value	Why
FEE PER EXECUTION	`0.010`	what the console suggests; keepers spend ~0.003 in group fees, so this leaves them ~0.007. The 0.004 minimum leaves only ~0.001 and cannot fund a machine.
FEE CEILING	`0`	off; only raise it if an upkeep is actually going unserviced
FUNDING	`0.03`	three runs at the suggested fee
IF A RUN IS MISSED	**Skip ahead**	see the box below

> **Leave it on Skip ahead for a first upkeep.** The alternative, *Catch up*, replays every missed interval at one fee each — the number of replays is bounded by how long it went unkept, not by your escrow. On this same deployment, upkeep 18 ran *Catch up* into a real outage, burned its entire escrow on 17 replays, and advanced 41 rounds against a 23,478-round backlog, then starved. On a short cadence, catch-up after any real outage cannot catch up. (Chapter 6 explains when *Catch up* is nonetheless the right choice.)

Lesson 3 — Read the cost before you sign

Check the **UP-FRONT COST** tile. With 0.03 funding it should read **0.0951 ALGO**:

Box deposit	0.0621	returned in full when you cancel
Escrow	0.0300	spent one execution at a time; remainder returns on cancel
Network fees	0.0030	three transactions, gone either way, even if the group fails

The console sets escrow equal to your funding, so 0.03 funding gives 0.0621 + 0.0300 + 0.0030 = **0.0951**. (Some of the project's own docs quote 0.0851 here — that total is for 0.02 of funding, not the 0.03 the form asks for; it is a documentation slip, not a console bug.)

> **Compare the tile against what your wallet actually asks you to approve.** This console figure was genuinely wrong once (reading 0.0741 against a real 0.0771 debit) and was caught by exactly this comparison. If the tile and the wallet disagree, that is a bug and worth more than the upkeep — report it.

Lesson 4 — Attest, connect, register

Tick **"I have tested this call against my own app and accept the risk."** It records human judgement and is deliberately *not* satisfied by the Test button passing. Arcron cannot know whether calling this method on a schedule is what you want.

In the CONNECT row, click your wallet and approve. **Pera** is used here, but Defly, Lute, Exodus, and Kibisis work the same way (and on LocalNet the console signs through KMD with no extension). Watch that the console reads your balance — it distinguishes *unread* from *zero*. Then press **Register upkeep** and approve.

You should land on `/u/<id>`, the upkeep's own page — not a confirmation panel. That page shows what it calls, its cadence, its next run, its escrow, its runway, and a plain sentence about what happens when the escrow runs out.

> **When will it actually run?** The project's only live keeper currently polls about **every 30 minutes**, so `RUNS` may stay `0` for up to that long even though everything worked — that is a keeper cadence, not a failure, and losing a race to another keeper is likewise free and normal. If you do not want to wait, you can run the first execution yourself from the upkeep page.

Lesson 5 — Clean up

On the upkeep's page, press **Cancel**. It refunds the remaining escrow plus the full 0.0621 box deposit to the account that registered. Cancelling is creator-only. (Leaving it running is also fine and mildly useful — it is one more upkeep on the network's uptime clock.)

If something goes wrong

- **Register stays disabled.** Both the attestation and a connected account are required; the hint beside the button says which is missing.
- **Pera shows a MainNet account.** It is still on MainNet — switch the node setting and reconnect.
- **"This is not the Arcron deployment."** The app id in the URL is not `769891898`. That panel is deliberate: anyone can deploy a contract with this ABI and box layout, so a look-alike shows the same registry and accepts the same form. Every money button stays disabled until you explicitly continue, and the id is not remembered. (Chapter 9 explains this quarantine.)
- **An execution fails.** Losing a race to another keeper costs nothing, and the chain rejects a failing transaction at validation rather than including it. Ordinary, not an error.

Chapter 5 — Integrating your own contract

For contract authors. By the end you will be able to write a hook Arcron can drive, authorize it correctly, and — most importantly — make it survive the ways integrations usually break.

Integration is *one method*. This chapter is everything else you need around it, in one pass. `examples/minimal_target.py` is a complete, compiling version of everything below; a test in the repo compiles it on every run, so it cannot rot.

Lesson 1 — The hook

Expose one NoOp ABI method that takes no arguments of its own:

```
@abimethod()
def run(self) -> UInt64:
    ...
```

Arcron calls it with the method selector as the only application argument. That is the simplest and most common shape — `tick()`, `publish()`, `distribute()`, `sweep()` in the repo are all built this way. A method taking arguments works too (the creator fixes the whole argument list at registration), but start here.

The design consequence: **your hook works from your own state.** It is not handed parameters, so whatever it needs to decide must already be on-chain when it runs. In practice that is a healthy constraint — it means anyone can verify what the scheduled call will do before it happens.

Lesson 2 — Authorization

Two choices, and most integrations should take the first.

Restrict to the keeper app. Arcron's inner call comes from the keeper application's *account*, so that is the sender to check:

```
assert Txn.sender == Application(self.keeper_app.value).address, "Only the keeper
 ↪  app"
```

Derive the address off-chain with `algosdk.logic.get_application_address(769891898)`.

Or leave it permissionless. Correct when the hook is idempotent and its timing is the only thing that matters: your contract then still works if Arcron disappears, and anybody can push it

along. It is the *wrong* default when the hook's effects depend on *when* it runs.

> **What restricting does not buy you.** The keeper app is permissionless, so "only the keeper app" means "only via a paid upkeep" — **not** "only by someone I trust." Anyone can point their own upkeep at your hook on the shortest interval, and pay the fees themselves. Read Lesson 4 before you rely on the check for anything that *counts*.

Lesson 3 — Make it durable (the part people get wrong)

Four rules, each learned the hard way.

Your hook is called whether or not there is work. Arcron calls on every cadence, forever. Make the no-op path cheap and make it *return*, not fail:

```
if self.pending.value == 0:
    return UInt64(0)    # right
    # assert False      # wrong - see below
```

A hook that fails stops being serviced. When a target rejects, the keeper bot backs that upkeep off exponentially (1, then 2, 4, up to 8 of the upkeep's own intervals, capped near an hour), and that state survives restarts. Failing costs the keeper nothing — Algorand rejects the transaction before it reaches a block — but it costs *you* the schedule. **Fail soft: record the problem in state and return.**

You have more opcode budget than you think. Budget pools across the app calls in a group, and an Arcron execution contains two (Arcron's call and the inner call to you). Measured: a method called directly gets ~684 opcodes at entry; called through Arcron it gets ~1,135 — roughly 1.66× — because it inherits the pool Arcron's own call contributed to.

Assume it may run more than once, in bursts. After an outage, `CATCH_UP` replays missed periods, so your hook may be called several times in quick succession. Make it idempotent, or make each call's effect depend only on current state.

Lesson 4 — Authorization is not authorization of cadence

This is the subtlest durability trap, so it gets its own lesson. The check everyone reaches for —

```
assert Txn.sender == Application(self.keeper_app.value).address, "Only the keeper
 ↪  app"
```

— proves the keeper application called you. It does **not** prove that your registered interval has elapsed, because registering an upkeep is permissionless. Anyone may point *their* upkeep at your hook, on the shortest interval, and pay the fees. Harmless for a hook whose effect depends only on current state (most of them). **Not** harmless for a hook that *counts, meters, or accrues* — a billing hook that advances a period on every call can be fast-forwarded by anybody willing to spend two minimum fees per call.

If your hook counts, enforce the interval yourself — and **return, do not assert:**

```
if Global.round < self.last_run.value + self.min_rounds.value:
    return self.count.value      # too soon: nothing to do, no rejection
```

```
self.count.value += 1
self.last_run.value = Global.round
```

> **Why return and not assert?** Under `CATCH_UP`, a backlogged upkeep calls again in the same round. An assert rejects that call, which fails the whole `execute`, which the keeper records as a failure and backs the upkeep off — until the schedule stops entirely. Returning refuses the *work* without refusing the *call*: the griefer still pays the fee and still moves nothing. `smart_contracts/subscription/` is the worked example that had both bugs in turn.

Lesson 5 — The pull pattern (the most useful technique here)

If you take one thing from this chapter, take this:

> **Do the accounting in the scheduled call. Let counterparties collect in their own transactions.**

```
scheduled_hook()   snapshot state, credit allocations, emit an event.
                   Move nothing. Call nothing.
claim()            the counterparty sends this themselves, and is therefore
                   always an available resource.
```

Two reasons it is not merely stylistic:

- **Resource availability.** An Arcron inner call reaches only what the keeper's transaction makes available, and nothing tells a keeper what your hook needs. A scheduled call that tries to pay an arbitrary account, read a balance, or call another app can fail because those resources are not available to it. (There is a mechanism to supply them — Lesson 6 — but pull sidesteps the whole question.)
- **Failure isolation.** A push payout to a closed or hostile account fails the *whole* execution, wedging the schedule for *everyone* your contract serves. Pull confines that risk to the one claimant.

`smart_contracts/rain/` (a scheduled prize draw) and `smart_contracts/subscription/` (a metered service) are both shaped by this.

Lesson 6 — Reaching resources your hook cannot name

A scheduled call can only touch what the executing transaction makes available, and Arcron stores no foreign arrays. It turns out it does not need to.

Resource references attached to the *keeper's* `execute` transaction flow **two levels down** — to Arcron's inner call and to your own inner transactions from it. And a keeper does not have to be told which ones you need: **simulation reports the resources a call would have required**, so a keeper simulates first, attaches what the simulation names, and then sends.

> **One nuance that will bite you if you skip it.** "The keeper fills in the references for you" is only true of a keeper that simulates and names them itself. The reference bot (`scripts/keeper_bot.py`) and the console's client (`js/src/keeper-txns.ts`) both do exactly that, covering up to six references. But `algokit-utils`' *default* resource populator caps at **four** direct account references and refuses a fifth. So do not assume

a keeper you did not write will fill the last two — a hook that touches five accounts, tested with a naive algokit client, will fail with `unavailable`. Still write the hook to reach for what it needs; still prefer pull.

Two real ceilings to respect:

- **Six references is the limit.** A hook needing more than six *cannot* be serviced by anyone — which is exactly what the pull pattern exists to sidestep.
- **Simulation sees state at simulation time.** A hook whose resource needs depend on state that changes between the simulate and the send can be mis-served. Keep what a scheduled hook touches predictable.

Lesson 7 — Calls with arguments

An execution carries up to three app args, counting the selector — enough for an ARC-4 method of arity two:

```
@abimethod()
def settle(self, market_id: UInt64) -> UInt64: ...
```

For anything wider, declare the arguments as a single struct or tuple (the trick ARC-4 itself uses at arg 15), which makes any arity reachable. **Every argument is fixed at registration.** If your hook needs a value that changes between runs, it must derive it from its own state, from a resource it pulls, or from the round — Arcron will not supply it, by design. (This is the "clock, not eyes" boundary; Chapter 11 draws it in full.)

Lesson 8 — An ASA bonus

An upkeep can pay a bonus in any asset **on top of** its ALGO fee, never instead of it:

```
register(..., fee_asset=<asset id>, asset_fee=<base units>)
opt_in_asset(mbr_payment, upkeep_id, asset)   # 0.1 ALGO, permanent
top_up_asset(upkeep_id, asset_funding)
```

Can you pay keepers *only* in your token? In effect, yes: set the ALGO fee at the 0.004 floor and it stops being a reward and becomes a cost reimbursement — at the floor an asset upkeep hands the keeper back exactly the ~0.004 ALGO it spent, and your token is the entire pay. What you cannot do is remove the ALGO altogether; that is Algorand's constraint, not Arcron's, since every transaction costs ALGO and no contract can price your token without an oracle.

Three things to know:

- An asset upkeep at the floor **only attracts keepers who want your asset** (they break exactly even in ALGO). Pay more ALGO if you want generic keepers too.
- The app must **opt in** before it can hold the asset — 0.1 ALGO of minimum balance, **permanent, non-refundable** (there is no opt-out).
- A keeper **not** opted in still executes, takes the ALGO fee, and forfeits the bonus, which stays in your escrow and comes back on cancel. This was verified on TestNet with a fresh, never-opted-in account (not just in mocks): upkeep 74 on app `769891898`, bonus asset `769987591`. The unopted execution (`ANSUPUK6VSXZ72IVP76ZDICGJ7NVVVV7BBKLNF25S3ZSFRDTTMWQ`) shows two inner transactions and an untouched bonus; a moments-earlier opted-in execution

(QQXW5G2OEJS5FXMA7M73YAEQFBOTR2RB3A7WUWAHVQ4YT6FTTYNA) shows the third transfer and the escrow falling by exactly the bonus.

Lesson 9 — The four things that will cost you an hour

None is Arcron's doing; all four are the toolchain, and all four look like your contract is wrong when it is not.

1. **Returning a computed value trips mypy before Puya sees it.** An ARC-4 field's `.native` is a `UInt64` at runtime but `Any` to mypy. Do not wrap in `UInt64(...)` (Puya rejects it). Bind through a typed local: `balance: UInt64 = box.value.balance.native; return balance`.
2. **A zero-argument method has no generated `Args` class.** Call it with no `args` at all.
3. **A create method with arguments is reached through `create` twice:** `app, _ = factory.send.create.create(args=CreateArgs(...))`.
4. **An inner payment with `fee=0` is paid for by the caller** (correct — it keeps the contract from spending on fees), but the calling transaction must cover both. Pass `extra_fee=AlgoAmount(micro_algo=1_000)`.

Lesson 10 — Test it, then test it again

> **Unit tests will not catch the things that break integrations.** `algorand-python-testing` mocks *record* inner app calls without executing them, and do not enforce minimum balances. A hook that fails when actually invoked, or an app that cannot pay what it owes, passes its unit tests and fails on a real chain. Anything depending on either belongs in a LocalNet end-to-end test.

```
algokit localnet start
fledge lanes run local        # the whole suite, including the worked demos
```

Lesson 11 — Close the loop: get something to actually execute you

Writing a hook is half of it; you have not integrated until a keeper has really called it. Two stages.

On LocalNet (make your own deployment, then drive it):

```
algokit localnet start
fledge run deploy-localnet     # deploys the keeper (add --with-pulse for the
↪  demo target)
# ... register an upkeep against your target ...
poetry run python -m scripts.keeper_bot --once --network localnet --app-id <app>
```

> **LocalNet is silent until you move the chain.** LocalNet runs in *dev mode*: a block is produced only when a transaction is sent, so rounds do not advance on a timer and an upkeep will look "not due" forever if nothing else is happening. Send transactions (or use the scripts' `network.wait_for_round` helper) to push rounds along. Anyone who deploys on LocalNet, starts a bot, and waits will correctly see nothing due — this is the surprise that gets everyone once.

On TestNet (register against the live keeper `769891898`):

```
cp .env.testnet.template .env.testnet          # add a funded throwaway
↪  DEPLOYER_MNEMONIC
poetry run python -m examples.register_upkeep
```

Then either wait for the project's keeper (the ~30-minute cron) to service it, or run your own keeper against it (Part III).

Chapter 6 — Scheduling and fees, in depth

For creators tuning an upkeep. By the end you will choose a missed-run policy and a fee ceiling deliberately, and budget an upkeep's runway correctly.

Two fields decide whether an upkeep survives the real world: `policy` and `fee_cap`. They were designed together, and this chapter treats them together.

Rounds are not a clock

A cadence is a round count, and a round is ~2.8 s nominally, ~2.66 s measured on TestNet. So "daily" means "every ~30,857 rounds," and the gap compounds:

Cadence	Rounds	At 2.8 s	At measured 2.66 s	Drift/cycle
hourly	1,286	1.0 h	~0.95 h (57 min)	~3 min
daily	30,857	24.0 h	22.8 h	~1.2 h
weekly	216,000	168.0 h	159.7 h	~8.3 h

A "daily" upkeep slides about **35 hours** against the calendar over thirty cycles, and which way depends on how busy the network is. Even the "hourly" row drifts: at the measured rate it fires every 57 minutes, which is why a nominal month of hourly runs costs more than the naive 720 executions (Chapter 10).

> **Arcron promises "not before this round." It never promises "at 09:00."** If a wall-clock moment matters, have the *hook* check the time and no-op when early, and schedule it often enough to catch the window. Do not set a cadence so tight that ordinary keeper lateness looks like a real condition; the minimum interval is 10 rounds, and the demos use a 30-round floor for anything that treats lateness as a signal.

Lesson 1 — CATCH_UP vs SKIP_AHEAD

`policy` is a **required** argument to `register`. There is no safe default, and the danger is that `CATCH_UP` is encoded as `0` — the value a caller passes *without deciding*. Decide.

- **`CATCH_UP` (0).** After an outage the upkeep stays due for every missed interval and catches up **one interval per call.** Right for work where every period genuinely owes something — a metering hook that bills per interval, a distribution that must not skip a recipient.

- `SKIP_AHEAD` **(1).** One execution advances to the first slot still ahead, keeping the schedule's phase; the backlog is dropped. Right for everything whose value is "it happened recently," not "it happened every time."

  The measured cost of getting this wrong. Upkeep 18 on `769891898` ran `CATCH_UP` into a real outage: it spent its entire escrow on 17 replays and advanced **41 rounds against a 23,478-round backlog**, then starved. Money gone, schedule still broken. On a short cadence, catch-up after any real outage *cannot* catch up. The console pre-selects `SKIP_AHEAD` for exactly this reason; anyone integrating directly should make the same choice on purpose rather than inherit it from a zero.

Lesson 2 — Fee escalation and the ceiling

`fee_cap` is the most one run may ever pay. Zero disables escalation entirely and the fee is always `fee_per_execution`. With a ceiling set, a neglected upkeep's fee rises **linearly from the base to the cap across one missed interval, then holds** at the cap. The effective fee is computed entirely from box state and paid to whoever executes.

Escalation exists to **clear a market**: when an upkeep goes unserviced, a rising fee recruits a keeper. Two consequences follow, and both matter:

- **With several keepers competing**, one takes the work early at a lower price and the ceiling is rarely reached.
- **With one keeper** — which is Arcron's situation today — the ceiling *is* the price, and the cadence is roughly half what you asked for, because a lone keeper is better off waiting for the fee to peak. So:

  Leave the ceiling at zero unless an upkeep is genuinely going unserviced. It buys reliability from a competitive keeper set and buys nothing from a single one. This is a "known and accepted risk" the project lists explicitly.

A subtlety the contract handles for you, and worth understanding: a ceiling does **not** raise the balance at which an upkeep goes dormant. When the escalated fee is more than the escrow holds, the fee **falls back to the base** so the upkeep stays executable by anyone, rather than stranding a ceiling's worth of escrow nobody can spend. Lateness only ever grows, so without that fallback an escrow that once fell below the escalated fee could never reach it again. **Budget your runway against the ceiling anyway**, since a late run can consume that much.

Lesson 3 — Escalation cannot be farmed

A natural worry: could a lone keeper deliberately let an upkeep fall behind and collect the ceiling over and over? No — and the reason is worth knowing because it is the contract's cleverest piece of economics.

Lateness is measured from `last_serviced_round`, not from the schedule, and that field jumps to "now" on every run. So **a replay of a backlog never escalates**: once an upkeep is behind, `next_execution_round <= last_serviced_round`, which the contract reads as "draining a backlog, pay base." A keeper collects the ceiling **once** per genuine fall-behind, then base for every replay behind it.

Measured before this guard existed: a patient keeper took **100% of a 400,000 µALGO escrow across 34 runs**, 33 of them at the ceiling. With the guard, only the first run escalates. This is pinned by a dedicated test and re-proven on a real chain.

Lesson 4 — Funding and runway

An upkeep is funded escrow, and executions are paid from it:

```
funded runs = balance / fee_per_execution     # with no ceiling
funded runs = balance / fee_cap               # with one, budget against this
```

At the 4,000 µALGO minimum fee, **1 ALGO buys 250 executions** — about 10 days of an hourly cadence, or 8 months of a daily one:

Escrow	Executions	Hourly	Daily
0.1 ALGO	25	~1 day	~25 days
1 ALGO	250	~10 days	~8 months
100 ALGO	25,000	~2.8 years	~68 years

Registering also costs the box deposit — 2,500 + 400 × (139 + len(encoded call_args)) µALGO, so **62,100 µALGO for a bare 4-byte selector** — and that comes back in full on cancel. Net, registering and cancelling costs only transaction fees.

Four operational facts to internalize:

- **Anyone can `top_up`.** Funding an upkeep that *already exists* is permissionless, so a counterparty who wants your schedule to keep running can pay for it. Only the creator can `cancel`.
- **Running dry is silent.** The upkeep goes dormant and resumes the instant someone tops it up. Nothing announces it.
- **A top-up does not reset lateness.** Funding a long-dormant upkeep is charged the ceiling on the very next run, because lateness is measured from the last *service* and a top-up is not one. (Resetting it would let anyone cancel escalation for one µALGO.)
- **Cancel when done.** A one-shot task that already ran keeps being called and keeps paying keepers to do nothing until you cancel.

To be warned before an escrow runs out, grep the health check rather than relying on its exit code (see Chapter 8 for why):

```
poetry run python -m scripts.keeper_bot --check --network testnet \
  --app-id <app> | grep -q starved && echo "top up something"
```

Part III — Running a keeper

Chapter 7 — What a keeper is, and where to run it

For anyone who wants to earn fees by keeping the network live. By the end you will have chosen a place to run a keeper and know exactly what its account needs.

> **Read this first, honestly.** Today every keeper running is the project's own, and no upkeep has been registered by a stranger — so at the current registry size, **a keeper's fees do not fund a host.** Run one now to *learn* the mechanics and to be the first independent keeper, not because it pays for itself yet. Chapter 10 has the arithmetic of when it would.

A keeper is a plain process

A keeper watches rounds and calls `execute` on due upkeeps. It holds a hot key, it needs to be up, and it earns fees. **Nothing about it is special to any one deployment** — the network is permissionless, so these are the options for anybody. The reference implementation is `scripts/keeper_bot.py`, a single Python process that services an entire registry.

The requirement is more forgiving than it looks. Upkeeps run on cadences of hours, and a neglected upkeep's fee escalates toward its cap, so a keeper that checks every fifteen minutes services a six-hour upkeep perfectly well. Latency only starts to matter when keepers compete for the same upkeep — which is not yet true on the live network.

How many keepers the network needs: two or three, not a crowd

This surprises people, so it is worth the arithmetic. One keeper is a loop over boxes; it does not shard. **Ten thousand upkeeps on an hourly cadence average 7.8 due per round** — one machine's work. Keeper count is a *liveness* question (is anyone watching?), not a *throughput* one, and the escalating fee exists to recruit the second keeper when the first stops, not to run an auction.

This is a direct lesson from prior art. Keep3r, on Ethereum at peak, had **six** distinct active keepers across the whole network. Designing for a large competitive keeper market is designing for something that has never existed. Two or three independent keepers is the target, and it is enough.

The options

Where	Cost	Uptime	Key lives	Effort
A server you already run (recommended)	nothing extra	continuous	on your box	one script
GitHub Actions cron	free to ~$115/mo by cadence	best-effort	repo secrets	uncomment a line
A small always-on host	~$2–5/mo	continuous	on that host	a container
A laptop	nothing	poor	on your laptop	one plist

A server you already run. If you have a VPS doing anything else, put the keeper on it — it is a small Python process and will not notice. The repo ships a packager that builds a ~392 KB tarball carrying no secrets (the mnemonic is typed on the host into a `640 root:keeper` file):

```
./deploy/vps/package.sh
scp /tmp/arcron-keeper.tar.gz <user>@<host>:/tmp/
ssh <user>@<host> 'sudo mkdir -p /tmp/arcron-install \
    && sudo tar -xzf /tmp/arcron-keeper.tar.gz -C /tmp/arcron-install \
    && sudo bash /tmp/arcron-install/deploy/vps/install.sh'
sudo -e /etc/arcron/keeper.env      # add KEEPER_MNEMONIC=
sudo systemctl start keeper-bot
```

A container (`deploy/Dockerfile`, `deploy/compose.yaml`) is the shape to reach for if you need minute-level polling. `restart: unless-stopped` covers reboots; `docker compose down` sends SIGTERM, which finishes the scan in flight so a redeploy never abandons a half-signed execution.

GitHub Actions (`.github/workflows/keeper-bot.yml`) runs `--once` on a schedule and is a *stopgap*, not the end state. Watch four things:

- cron granularity is ~5 minutes and best-effort (GitHub delays and may drop runs);
- **each run is a fresh process with no disk**, so the backoff state does not persist — a persistently failing upkeep is retried every run (see Chapter 8);
- the mnemonic lives in repository secrets;
- **a scheduled workflow is disabled after 60 days without repository activity**, so a quiet month turns the keeper off silently.

The cost is billed per minute on a private repo, and a keeper runs constantly:

Cadence	Runs/month	Minutes	Cost beyond the 3,000-min allowance
every 5 min	8,640	~17,300	~**$115/month**
every 15 min	2,880	~5,800	~$22/month
every 30 min	1,440	~2,900	free (consumes the whole allowance)

> **Match your keeper's cadence to the upkeeps it serves.** A half-hourly keeper suits work measured in hours. Point it at a ten-minute upkeep and it is three intervals late, and with `CATCH_UP` it replays every missed interval at one fee each. If you need minutes, run something that polls in minutes.

The second keeper, and why it is not just a backup

`.github/workflows/keeper-bot-2.yml` is a second keeper on the *same* cron as the first. It exists to make Arcron's economic claim — that competition holds the fee below the ceiling, and that losing a race costs nothing — *actually happen* on a real chain, where neither had ever been observed.

> **An offset schedule does not race — it queues.** Two keepers thirty minutes apart never contend: the first takes every due upkeep and the second arrives to an empty registry. That looks like redundancy and is a queue.

So both workflows pass `--align 120`, which holds the first scan until the next whole two-minute mark in UTC. Runner clocks are NTP-synced, so an absolute instant is the one thing two machines that have never met can agree on. They then scan in the same round window and reach for the same upkeep — which is what a race *is*. The second keeper must sign from a **different account** (`KEEPER_2_MNEMONIC`); one account cannot race itself. Nothing about the barrier is specific to GitHub — a VPS keeper joins it with the same flag.

What the account needs

A keeper pays 3,000 µALGO per execution and collects the upkeep's fee, so it is profitable as long as fees exceed costs. It **refuses to start below 103,000 µALGO** — 100,000 to keep the account, plus one execution. Use an account that holds no more than it needs: it is a hot key on an unattended machine whose whole job is to spend small amounts constantly.

Chapter 8 — Operating a bot

For keeper operators. By the end you will run a keeper in a loop, read its logs, health-check it from outside, and know exactly what a race and a backoff look like.

Before the first run

The bot needs three things set, and omitting any of them is the usual reason a first run does nothing:

- **Python 3.13** (never 3.14 — a dependency has no wheels for it), in the Poetry venv (`poetry install`).
- **A network and its config.** Pick the chain with `--network` (or `ARCRON_NETWORK`); it loads `.env.<network>` and checks the node's genesis id.
- **A signing key and the app to service.** `KEEPER_MNEMONIC` (else `DEPLOYER_MNEMONIC`) signs and pays the fees; **the app id is required** — pass `--app-id <id>` or set `KEEPER_APP_ID`. There is no built-in default deployment.

The commands

```
poetry run python -m scripts.keeper_bot --network testnet --app-id 769891898
 ↪  # loop
poetry run python -m scripts.keeper_bot --network testnet --app-id 769891898
 ↪  --once     # single scan (cron)
poetry run python -m scripts.keeper_bot --check --network testnet --app-id
 ↪  769891898     # probe, signs nothing
```

Reading the logs

`--log-format json` (the container default) emits one object per line. The line you will care about months later is `executed`:

```
{"event": "executed", "round": 66629379, "upkeep_id": 9, "target_app": 1043,
 "fee_collected": 4000, "escrow_remaining": 8000, "next_due_round": 66629389,
 "tx_id": "F724IJ7A...UC6A"}
```

It carries the round, the fee collected, the escrow left, and the transaction id, so any claim can be checked against the chain. Use `--log-format text` for a human at a terminal.

Health checks from outside

`--check` reads the registry and exits **without signing anything**, so it works as an external probe — you do not need the keeper's account or its cooperation. It draws a distinction that matters, and the exit code encodes it:

- **Stalled** — an upkeep is funded, due, and nobody came. This is a *keeper* problem. `--check` **exits 1**.
- **Starved** — an upkeep's escrow has fallen below one fee, so no keeper *can* execute it. This is the *creator's* problem, not a keeper's, so `--check` **exits 0**.

  Do not page on `--check`'s exit code for starvation. Blaming keepers for a starved upkeep would make the signal useless, so a starved upkeep exits zero. To be alerted when an escrow runs dry, grep the output: `... --check ... | grep -q starved && echo "top up something"`.

Knowing it is still alive

A keeper fails silently in two ways, and both take the network down with it.

It dies. Nothing on-chain says so; upkeeps just quietly pile up as due. Every twenty scans (and on every `--once` run) the bot emits a `heartbeat`. **Alert on its absence, not its content** — a heartbeat that stops is the signal.

It runs out of ALGO. This is nastier, because a keeper earns fees into the same account it spends from: self-sustaining while the registry is busy, stuck the moment it is empty. So the balance is checked before the first scan and at every heartbeat. Below 103,000 µALGO it refuses to start and exits 2; below `--min-balance` (default ~100 executions of headroom) it warns each heartbeat and keeps working.

Races, and why losing is free

Multiple competing bots are safe: the contract re-checks due-ness atomically, so **exactly one keeper is paid per due round, and the loser pays nothing.** Algorand rejects a failing transaction at validation rather than committing it, so it never enters a block and no fee is charged — unlike an EVM revert, which still burns gas.

This is not inferred, it is measured three ways: a losing `execute` broadcast straight to algod (its balance unchanged), a race between two real bots on a shared barrier (the loser's transaction absent from any indexer, its balance moved by exactly zero), and the ordinary in-pool race. A race leaves almost no trace — only the winner's transaction exists — so the **losing keeper's own log is the record**, written to carry everything an outsider needs to verify it against the chain:

```
{"event": "race_lost", "round": 66703234, "upkeep_id": 75,
 "winner": "NUGVPQGZ...QMBVU", "won_at_round": 66703238,
 "fee_forgone": 4000, "spent": 0, "registry_advanced": true,
 "tx_id": "KXTAGVSRJAYXTUGRGA5VY73SLRRH2YGUKIB7YIFOEUBWM4P7XDXQ"}
```

`spent: 0` is the whole argument for running a keeper: **losing costs nothing.** To produce a race on purpose rather than waiting for the schedule:

```
poetry run python -m scripts.keeper_race --network testnet \
    --app-id 769891898 --target-app 769891902
```

It registers a fast upkeep, starts two real bots against the same barrier, checks the outcome against chain data, and **exits non-zero if the two keepers did not actually collide** — because a run in which they politely took turns proves nothing.

Backoff: a failing target, versus a lost race

The bot separates the two, because they mean opposite things.

- **A failing upkeep** (a target that rejects the call) **backs off exponentially** — the wait doubles in the upkeep's own intervals up to 8×, capped near an hour (1,286 rounds) in absolute terms. That state survives restarts, so a `--once` cron does not re-attempt a doomed upkeep every run. A success resets it to zero. Once you have fixed a target, `--retry-now <id>` clears one upkeep's backoff and `--clear-backoff` clears them all.
- **A lost race never backs off.** Another keeper getting there first is the common, healthy, free case; a keeper that stopped trying everything it lost would service less and less of the registry.

  The GitHub Actions exception. Backoff persistence lives in a state file (`XDG_STATE_HOME`, or `--state-file`, or `--no-state` to disable). A GitHub Actions run is a fresh process with no persistent disk, so backoff does *not* carry between runs there — a doomed upkeep is re-attempted on every scheduled run. That is one more reason the cron keeper is a stopgap.

The two signals separating a failing target from a lost race are not equally trustworthy. The error text arrives first, but a target has some say in it (algod disassembles the failing program into the message). The registry itself is the honest signal: if the upkeep's box moved on between the scan that picked it and the call that failed, somebody executed it, and nothing a target writes can fake that.

Announcing what happened

A network whose work is invisible looks dead even when it is fine. `scripts/notifier.py` watches the registry and says what changed — to a Discord webhook or to the terminal:

```
poetry run python -m scripts.notifier --network testnet          # prints here
DISCORD_WEBHOOK_URL=https://... poetry run python -m scripts.notifier --network
 ↪  testnet
```

Three properties worth knowing: it **holds no keys and cannot sign** (enforced by a test that fails if anything key-shaped appears in the module); it **needs no indexer**, deriving "which keeper" by reading the few blocks between its scans; and **restarting is quiet** (the last announced state is persisted, so a restart replays nothing). It surfaces *failures* deliberately — an upkeep out of funds, or funded and due with nobody servicing it — because saying so builds more trust than a feed of good news.

Part IV — Trust, security, and economics

Chapter 9 — The security model

For anyone deciding whether to escrow real value. By the end you will know exactly who can touch your money, what the one real admin power is, and which risks the project has accepted on purpose.

Arcron is unaudited. What follows is the project's own analysis — written down so it can be argued with rather than taken on trust. Where this book adds a check of its own, it says so and says what kind of check it was.

Who can do what

The three parties are the same ones from Chapter 2 — creator, keeper, target app — and the boundaries between them are the whole safety story. The rules that protect money, specifically: a keeper can never take more than the fee the box records; a creator can never touch another creator's upkeep; a target can never reach the keeper's funds or re-enter Arcron. Everything below is how those hold.

The one real admin power: upgradeable until frozen

This is the biggest thing to understand, and the project says so first: **whether there is an admin key over your escrow depends on one flag.**

A deployment starts **unfrozen**. Until its creator calls `freeze`, that creator — and only that creator — can replace the app's programs with `update`, and thereby reach every escrow in the app. They could redirect payouts, raise fees, or drain escrow. No statement of intent removes that power; the honest way to describe it is that *"no admin key" describes the deployment you are heading towards, not the one in front of you.*

`freeze` gives that power up **permanently and one-way**: nothing sets `frozen` back to 0, and no later call can restore an update path (the only call that could is an update, which is now refused). And the flag is global state, so the promise can be **checked rather than believed**:

```
poetry run python -m scripts.govern status --network testnet --app-id <id>
poetry run python -m scripts.verify_build --network testnet --app-id <id>
```

The first says whether the creator can still change the rules; the second says whether the deployed bytecode is the source it claims to be. Together they are the whole trust question.

> **Freezing does not remove risk; it exchanges one risk for another.** An *unfrozen* deployment can be repaired by someone who could also rob you. A *frozen* one can be robbed by nobody and repaired by nobody — its safety rests entirely on the bytecode

being right the first time. Read alongside the MainNet gate (self-review, no paid audit), a frozen MainNet deployment is three things at once: no admin key, no third-party review, and no way to patch. Each is defensible; the combination is the actual risk.

Why does the window exist at all? Because being unable to fix a bug is expensive while nobody depends on the deployment yet. Two earlier deployments were abandoned rather than repaired, stranding **243,000 µALGO** of box deposits and forcing every creator to cancel and re-register by hand. `DeleteApplication` is refused always, frozen or not, because deleting an app with escrow would strand every µALGO.

What "alpha" means, and the release stages

The deployment's stage is not just a colour word. The project runs a stage ladder — **alpha -> beta -> rc -> mainnet** — and the current TestNet deployment is **alpha**, which carries a specific promise and a specific non-promise:

- **alpha / beta / rc** live on TestNet. An **alpha** app id may be replaced for any reason; expect to `cancel` and re-register if it is. Beta and rc add sustained-uptime and self-audit requirements before MainNet is considered.
- **mainnet** is gated on the 1.0 contract staying *unchanged* for a sustained period (any struct change restarts the clock), a continuously serviced dogfood with the notifier's record as evidence, and a rigorous self-audit — **no paid audit** — behind a 2-of-3 multisig.

So "alpha, unaudited" on the title page is a contract, not a disclaimer: use only `769891898`, and treat it as replaceable.

The threat model, checked

The project enumerates adversaries and how each is stopped. The claims below were also read against the contract source and its compiled bytecode while this book was compiled — a source-and-bytecode read, **not an audit**.

An adversarial keeper.

- *Take more than the fee?* No. The fee is computed entirely from box state and paid to the caller. **Confirmed** by reading the payout paths.
- *Serve slowly to be paid more?* With a fee ceiling, yes — that is escalation working as designed, bounded by `fee_cap`. What it **cannot** do is farm the ceiling off a backlog: a replay never escalates (Chapter 6, Lesson 3). Measured before the guard: 100% of a 400,000 µALGO escrow across 34 runs; after it, the first run only. **Confirmed** by a dedicated test and by hand.
- *Drain one upkeep to pay another?* No. Each box carries its own balance, checked before payment; the app's *spendable* balance always covers the sum of every escrow. **Confirmed:** the refund on cancel exactly matches the escrow plus the released deposit, and inner-transaction fees are set to zero (fee pooling), so the escrow is never touched for fees — the keeper's own transaction group pays them.

An adversarial creator.

- *Strand the app account?* No. `register` collects exactly what the box costs (derived from the encoded box, not restated) and `cancel` returns exactly that.
- *Register an upkeep that traps its own funds?* Not any more. Three states used to register happily and then fail forever — an argument list longer than the fan-out, a `fee_cap` the escrow could never reach, and a `fee_asset` with a zero bonus. All three are now rejected at registration, and escrow always leaves by `cancel` if nothing else.

A malicious target app.

- *Re-enter Arcron?* No — the AVM refuses outright (`attempt to re-enter <app>`), and independently the contract writes box state before submitting any inner transaction. Two lines of defence.
- *Spend the keeper's ALGO?* No. Arcron's inner transactions carry a zero fee and draw on the group's pooled fee, which the keeper sized; a target's own inner transactions are paid by the target.

> **What a read for this book found.** While compiling this book, its author read the contract source and compiled bytecode looking specifically for a path that loses money, a way to lock an upkeep's funds without the creator's consent, and a griefing win. **That pass found none.** The refund accounting is conservative, the one dangerous multiply (fee escalation) is bounded by the input caps and cannot overflow, and a creator can *always* cancel and recover — even against a hostile bonus asset, because the ASA transfer is best-effort while the ALGO refund is not. This is a source read, **not an audit**, and the project remains unaudited; treat it as one more reason to check for yourself, not a clean bill.

The console's one defense: the quarantine

The contract is permissionless, so anyone can deploy a look-alike with the same ABI and box layout. The console's address is therefore a *security property*: a link carrying a different app id could point a stranger at a hostile clone that shows the same registry and accepts the same register form.

The defense is **quarantine**. A link naming an app that is not the published one lands as *foreign*, and three things follow, none optional: every money button is dead (`canCommitMoney` refuses independently of the display logic), the id is never written to browser memory so the poison cannot outlive the visit, and the console says so, names both ids, and offers one click back. LocalNet is treated as *unverifiable* rather than *foreign* — there is no published deployment there and the node is your own machine, so a link cannot aim it at anything an attacker controls.

> **A caveat this book adds.** The console's *primary* mitigation — that the `?app=` parameter is inert outside developer mode — is weaker than it reads, because developer mode is itself switched on by a URL parameter (`?dev=1`). The quarantine still holds (a foreign app's money buttons stay dead behind an explicit "continue anyway" click), so this is defense-in-depth erosion, not a bypass — but treat the quarantine, not the inert parameter, as the real barrier, and never click "continue anyway" on an app id you did not choose.

Known and accepted risks

These are real, understood, and shipped anyway — the honest list, condensed:

- **A lone keeper is paid the ceiling.** With no competition, escalation is not a worst case; it is the price. Mitigation: the default is no escalation (`fee_cap = 0`).
- **A top-up does not reset lateness**, so funding a long-dormant upkeep is charged the ceiling on its next run. The console warns.
- **An upkeep can be stranded by its own target.** If a target becomes unexecutable and the creator is gone, the escrow is stranded (only the creator can recover it). Prefer immutable targets, or ones you control.
- **A refund can fail if the creator's account is empty**, because Algorand rejects a payment leaving the receiver below the 100,000 µALGO account minimum.
- **Overpaid box deposit is not returned.** Send the exact amount; the console computes it.
- **Registry spam degrades keepers.** Box deposit is refundable, so a spammer's real cost is only transaction fees and locked capital; a keeper that cared would cache boxes.

Reporting a bug

A live-funds vulnerability goes to a **private draft security advisory**, not a public issue; `SECURITY.md` is the authoritative policy. Anything already public can be a normal issue.

Chapter 10 — The economics, honestly

For anyone weighing Arcron against the alternatives. By the end you will know where it wins, where it loses, and the one structural tension the project has not resolved.

It is cheaper than a paid host, not cheaper than free

Chapter 2 gave the table; here is what it means. Against **paid** hosts you would otherwise run a bot on, Arcron **at the 4,000 µALGO floor** is several times cheaper — the floor row is about a seventh of the cheapest paid host ($0.28 vs $2.02). At the fee the console actually suggests (10,000 µALGO, ~$0.65/month) the gap is smaller — roughly three times cheaper — but still real. Against the **free** options (AWS Lambda + EventBridge, GitHub Actions in a private repo), it is not cheaper at all, and the project says so. The ratio is also a bet on the ALGO price: it moves *against* Arcron precisely when Algorand succeeds, reaching parity with a ~$2/mo host around ALGO = $0.70, a price ALGO has traded above within the last two years.

> One number the project's docs and this book both flag as soft: "7.7× cheaper than the cheapest paid host" does not reproduce from the printed table ($2.02 / $0.28 = ~7.2×), and it silently uses the *floor* fee, not the suggested one. Recompute against the fee you set.

Where running your own bot wins

Above about **26 concurrent hourly upkeeps**, running your own bot is cheaper, because one process services any number of targets from one key. The reference bot is a single process servicing the whole registry, so "ten contracts means ten bots" is false. The asymmetry that survives is narrower and real: **no hot key, and no operational attention.** That is worth something, and it is not a process count.

The uncomfortable structural finding

This is the part the project flags as not comfortable, and it is the most important thing in this chapter.

At the 4,000 µALGO floor a keeper nets ~1,000 µALGO per execution, so **one keeper needs roughly 77 concurrent hourly upkeeps to fund a $5 host.** But a *creator's* crossover — the

point where self-hosting beats paying Arcron — is around **26**. Those numbers are the wrong way round:

Fee	Creator pays/mo	Creator crossover	Keeper break-even
4,000 µALGO (the floor)	~$0.28	~26	~77
10,000 µALGO (suggested)	~$0.65	~9	~11
20,000 µALGO	~$1.31	~4	~5

Between 26 and 77 upkeeps, a creator self-hosts anyway, and below 26 the aggregate fees cannot fund the server Arcron says it replaces. **The floor fee is below the cost of supplying it.** Raising the fee closes the gap — around 10,000 µALGO the two converge and the network pays for itself, still several times cheaper than a paid host. That is the sustainable operating point, and it is *not* the one the minimum advertises. The contract half-admits this already: *"A creator who wants keepers who do not care about their token should set a fee above this floor."*

> **The takeaway for a creator:** do not register at the floor and expect a stranger to keep your upkeep alive for free. Price it at the point where a keeper actually profits (~10,000 µALGO for an hourly upkeep, which is what the console suggests), or run the keeper yourself. This is an economics problem, not a safety one — but it is the one to understand before you rely on the network.

Rounds drift, and that costs you too

Because a cadence is a round count and rounds run slightly faster than nominal, an "hourly" upkeep fires every ~57 minutes and slides ~36 hours against the calendar over a month — which also means it fires *more often* than 720 times, so it costs more than the naive 2.88 ALGO/month. Budget against your real cadence, not the nominal one.

Chapter 11 — Is a keeper network the right idea?

For anyone judging the thesis, not the code. By the end you will understand the strongest form of the argument, the boundary the design refuses to cross, and the single test the project has staked itself on.

The argument that does not need agents

Strip away the hype and the claim is small and durable: *a smart contract cannot wake itself up, and Algorand has no productized way to give it a heartbeat.* That is true today, checkably (no ARC, no shipped fully-permissionless alternative), and it survives being wrong about everything else.

The argument that does need agents, at its real strength

The payments world is about to have a "pay" verb for autonomous agents (x402 and the agent-to-agent work) and no "wake up" verb — x402's own spec puts recurring payments and open-ended allowances explicitly *out of scope.* So there is a real gap. But be precise about how much it proves:

- **The weak form is wrong.** "Agents need scheduling, so they need Arcron" does not follow. An agent alive enough to hold funds and make decisions is alive enough to call its own contract, and cheaper.
- **The strong form is the interesting one.** Arcron wins when the schedule should **outlive the agent that created it** — when autonomy should not be only as durable as somebody's running process. That is liveness that survives its author, and no agent framework provides it, because every framework assumes the agent is running.

Whether anyone wants that is stated, honestly, as an open question rather than a claim.

The boundary the design refuses to cross

Arcron is **the clock, not the eyes.** It will not let a keeper supply *data*, and this is a deliberate closed door, not a missing feature (issue #22, closed not deferred). The reasoning is worth internalizing because it defines the product:

> Letting a keeper choose *what a contract is told* makes every keeper a trusted party. That is an oracle network, a different product. Declaring which *resources* a call may touch is

> safe, because the creator still fixes what is called; letting a keeper supply *data* inverts the one guarantee Arcron makes.

The supported answer for data-driven automation is **oracle pairing**: a reporter pushes values into an oracle, an Arcron upkeep triggers `settle()` on a cadence, and settlement reads the stored value. Arcron supplies the timing guarantee — settlement cannot be stalled, delayed, or selectively timed by an interested party — and nothing else. One case needs no oracle trust at all: a **staleness check** that compares a feed's last-updated round against the current round, because comparing round numbers cannot be lied to.

For the same reason, **keeper staking (#15) is closed too**: a keeper has exactly one action (`execute`), a wrong execution is impossible (the call is fixed), a failed one is already free, and not executing is not an offence — so a bond would have nothing to slash and would add an owner-shaped thing to an ownerless contract.

The lessons from those who tried

Every predecessor failed at the same seam — adoption, not engineering. AlgoRhythm stopped exactly at the incentive design. BiatecCron shipped and ran three tasks in two years, all its author's own. Keep3r's active keepers peaked at six and most of its jobs were never worked. The engineering is not the hard part; **being a well-built thing someone needs** is.

The one test that settles it

The project has written down the falsifiable version, and it is refreshingly narrow:

> If this is real infrastructure, somebody outside the project registers an upkeep for something they actually wanted scheduled, within a few months of it being visible. If a year passes and every upkeep is still theirs, the design was fine and the demand was not there.

That is the number that settles it — not keeper count, not throughput, and nothing Ethereum measures. As of this writing, the count of upkeeps registered by strangers is **zero**, and the project says that louder than any critic would. The mechanism holds up. Whether the world wants it is genuinely unknown, and that honesty is the most trustworthy thing about the project.

Part V — Reference

Look-up material for building against Arcron. Everything here is stated elsewhere in the book; this part collects it in one place. Where a figure is a measured claim, treat it as checkable against the live chain.

Appendix A — The public API

All methods are ARC-4 ABI methods on the keeper app (`smart_contracts/keeper/contract.py`). The exact signatures matter, because a selector is `sha512_256(signature)[:4]`; a table of parameter *names* is not enough to compute one, and getting it wrong yields `logic eval error: err opcode executed` with no mention of the method.

```
register(pay,pay,uint64,byte[][],uint64,uint64,uint64,uint64,uint64,uint64)uint64
execute(uint64)uint64
top_up(uint64,pay)uint64
cancel(uint64)uint64
opt_in_asset(pay,uint64,uint64)uint64
top_up_asset(uint64,axfer)uint64
freeze()void
update()void
```

> Note `opt_in_asset` takes the asset as a plain `uint64`, **not** the ARC-4 `asset` reference type. The natural guess is wrong. The machine-readable source of truth is the ARC-56 spec at `smart_contracts/artifacts/keeper/Keeper.arc56.json`.

The methods:

Method	Callers	Purpose
`register(...) -> uint64`	anyone	Create an upkeep; returns its id. Full 10-argument signature in the code block above; each argument is detailed under *register preconditions* below.
`execute(upkeep_id) -> uint64`	anyone	Fire a due, funded upkeep; pays the caller the effective fee; returns the next due round.
`top_up(upkeep_id, funding_payment) -> uint64`	anyone	Add escrow; returns new balance.
`cancel(upkeep_id) -> uint64`	creator only	Delete the upkeep; refund escrow **plus** box deposit **plus** any unspent bonus.
`opt_in_asset(mbr_payment, upkeep_id, asset) -> uint64`	anyone	Let the app hold an upkeep's bonus asset. 0.1 ALGO, permanent, non-refundable.

Method	Callers	Purpose
`top_up_asset(upkeep_id, asset_funding) -> uint64`	anyone	Add to an upkeep's ASA bonus escrow.
`freeze() -> void`	creator only	Give up the update path permanently.
`update() -> void`	creator only	Replace the programs. Refused once frozen.

The register preconditions that are in no signature — each fails as a bare `assert` that names nothing:

- **Both payments go to the keeper application's own account** (the address derived from its app id), not to the creator and not to a keeper.
- **Both payments must be sent by the same account that sends the app call.** A third party cannot fund somebody else's *registration*. (`top_up` is the opposite: funding an upkeep that already exists is a permissionless gift.)
- **Group order is `[mbr_payment, funding_payment, app call]`.**
- **The box deposit is a minimum, not exact.** Overpaying is accepted and not refunded — pay the formula.
- **The funding payment must cover at least one execution** at the price the upkeep can be charged: `fee_cap` when a ceiling is set, else `fee_per_execution`.
- **The call must carry a box reference for `b"u" + itob(n)`**, where `n` is the app's global `next_upkeep_id`. Read that global first to predict the id (a typed algokit-utils client does this for you).

The execute preconditions:

- **Your `execute` transaction must carry the box reference for `b"u" + itob(upkeep_id)` and a foreign-app reference to the target.** Without the box: `invalid Box`; without the app: `unavailable`. Arcron spends two of the eight reference slots on your behalf but does not attach these two for you.

Constraints asserted on-chain:

- `10 <= interval_rounds <= 1,000,000,000` rounds.
- `4,000 <= fee_per_execution <= 1,000,000,000` µALGO.
- **App-arg count 1–3** (`0 < count <= 3`, counting the selector); the encoded argument list must be `<= 1,024` bytes. (The lower bound is on the *count*, not the byte length: an empty argument list is rejected on count.)
- `policy` is `CATCH_UP` (0) or `SKIP_AHEAD` (1); `fee_cap` is either 0 or between `fee_per_execution` and 1,000,000,000 µALGO; `fee_asset == 0` or `asset_fee > 0`.
- Executions are NoOp inner app calls carrying every stored app arg.
- The ASA bonus is paid **on top of** the ALGO fee, never instead of it.

Appendix B — The Upkeep box encoding

Each upkeep is one box, named `b"u" + itob(upkeep_id)` (9 bytes), holding an ARC-4 head/tail encoding of the `Upkeep` struct. The head is **always 130 bytes**; the contract always writes 130 as the tail offset at bytes `[40:42]`.

Bytes	Field
`[0:32]`	creator address
`[32:40]`	target app id
`[40:42]`	offset to the `call_args` tail (always 130)
`[42:50]`	interval_rounds
`[50:58]`	next_execution_round
`[58:66]`	fee_per_execution
`[66:74]`	balance
`[74:82]`	times_executed
`[82:90]`	policy
`[90:98]`	fee_cap
`[98:106]`	last_serviced_round
`[106:114]`	fee_asset
`[114:122]`	asset_fee
`[122:130]`	asset_balance
`[130:]`	tail: ARC-4 `byte[][]`

> **The tail rule that will bite you.** The tail is a `uint16` count, then one `uint16` offset per argument, then each argument as a `uint16` length followed by its bytes. **Every offset is measured from just after the count, so add 2 before indexing into the tail.** Omitting the +2 does not raise — it yields a plausible wrong value (one real box decodes to `["0004"]` instead of `["40d7be68"]`). Both reference decoders reject any box whose head is not 130 bytes rather than reading past the end of a shorter, older box.

Box deposit (minimum balance): 2,500 + 400 × (139 + len(encoded call_args)) µALGO — **62,100 µALGO for a bare 4-byte selector.** Fully refunded on cancel.

Global state: `next_upkeep_id` (the id `register` assigns next — read it to predict your box name) and `frozen` (0 = still updatable, 1 = frozen; an app predating governance carries no `frozen` key and reads as frozen).

Reference decoders, pinned to the same recorded box so they cannot drift:

- Python — `scripts/keeper_bot.py::_decode_upkeep`
- TypeScript (the console imports it) — `js/src/upkeep.ts`

Appendix C — Command cheat-sheet

Build, test, CI (`fledge` is the project's task runner; prefer it over calling tools directly):

```
fledge lanes run ci          # build + unit tests + strict spec check (keep
↪  green)
fledge lanes run local       # ci + the LocalNet end-to-end (needs algokit
↪  localnet)
fledge lanes run endurance   # build + tests + spec + a soak + a scenario (a
↪  longer suite)
poetry run python -m smart_contracts build      # Puya compile + typed clients
poetry run pytest tests/ -q                       # unit tests (mocked chain)
specsync check --strict                            # spec-drift check
poetry run python -m scripts.keeper_e2e --network localnet   # full e2e
```

The console:

```
cd web && bun install && bun run ng serve     # dev server at :4200
bun test                                       # console unit tests
fledge run web-render                          # the rendered-page measurement
↪  audit
fledge run web-build-hosted && fledge run web-verify-hosted   # hosted build
```

Register / operate (choose the network with `--network` or `ARCRON_NETWORK`; `--app-id` is required — there is no default deployment):

```
poetry run python -m scripts.keeper_bot --network testnet --app-id 769891898
↪  # keeper loop
poetry run python -m scripts.keeper_bot --once --network testnet --app-id
↪  769891898  # single scan
poetry run python -m scripts.keeper_bot --check --network testnet --app-id
↪  769891898 # health probe
poetry run python -m scripts.keeper_race --network localnet
↪  # prove a race
poetry run python -m scripts.notifier --network testnet
↪  # registry watcher
cp .env.testnet.template .env.testnet          # then, for a worked registration:
poetry run python -m examples.register_upkeep
```

LocalNet:

```
algokit localnet start        # start the local chain
```

```
fledge run deploy-localnet     # deploy the keeper (add --with-pulse for the pulse
↪  target); idempotent
algokit localnet reset         # empty chain
```

Rules of the road: Poetry venv, Python **3.13** (never 3.14 — coincurve has no wheels). `.env.<network>` holds per-network config and is gitignored; never commit a mnemonic. The TestNet deployer is a throwaway and must never be reused on MainNet. LocalNet is dev mode — rounds only advance when a transaction is sent.

Appendix D — Deploying and governing a deployment

Deployment is deliberate on every network — nothing is automated.

```
fledge run deploy-localnet      # LocalNet only
fledge run deploy-testnet       # needs .env.testnet with DEPLOYER_MNEMONIC
fledge run deploy-mainnet       # needs .env.mainnet AND ARCRON_ALLOW_MAINNET=1
```

Two program pages. The contract compiles to just over 2,048 bytes, so it allocates an extra program page. The extra page costs the **creator (deployer) account** 100,000 µALGO of minimum balance, permanently, and the app account has its own 100,000 µALGO base minimum on top — so budget **0.2 ALGO** locked total, and note *which* account: the page MBR is locked on the deployer, not the app. Pages cannot be added by an update — it is create-only.

Governance lifecycle (`govern`):

```
fledge run govern -- status  --network testnet --app-id <id>   # frozen? which
↪  bytecode?
fledge run govern -- update  --network testnet --app-id <id>   # replace programs
↪  (unfrozen only)
fledge run govern -- freeze  --network testnet --app-id <id>   # give up update,
↪  forever
```

`status` prints the creator, the program sizes, the **combined** approval+clear `sha256`, and `frozen`. Always compare the *combined* digest — an approval-only hash would let a hostile clear program ship beside an honest approval. A pre-governance app prints `frozen absent`, which is the *stronger* guarantee (no update path at all).

Multisig, for MainNet. Set `ARCRON_MULTISIG_THRESHOLD` and `ARCRON_MULTISIG_ADDRESSES` and the creator becomes a multisig; `deploy.py` refuses to run from a single key. `govern update`/`freeze` then write an unsigned transaction for holders to sign wherever their keys live — always `show` a file before you `sign` it:

```
fledge run govern -- update --network testnet --app-id <id> --out update.json
fledge run govern -- show   --file update.json --app-id <id>
SIGNER_MNEMONIC="..." fledge run govern -- sign --file update.json --app-id <id>
fledge run govern -- submit --file update.json --app-id <id>
```

The MainNet plan is **3 keys, threshold 2** — one may be lost and one compromised without losing control. Member order is part of a multisig address, so the same keys in a different order are a different account. Post-quantum Falcon accounts cannot be multisig members (their address is a

hash, not a curve point), and `scripts/multisig.py` refuses them with a real curve-membership test.

Verifying a deployment you did not make: `verify_build` rebuilds from the working tree and compares the compiled **bytecode** (not the TEAL text, which loses comments on assembly) against what algod reports for that app.

Appendix E — The design decisions, in brief

Why the system is shaped the way it is, distilled from the project's design docs.

1.0 scope (decided 2026-08-24). The `Upkeep` struct cannot change in place — an update replaces code, not box shape — so a struct change means a new app id, an empty registry, and every creator re-registering by hand. Four struct-touching features were therefore batched into one final release (per-upkeep catch-up policy #7, fee escalation #14, resource declaration #8, ASA-fee capability #9), and the surface is then **frozen**.

Why CATCH_UP and SKIP_AHEAD had to ship with escalation. Combining catch-up replay with escalation measured *from the schedule* would make every replay pay the escalated fee (measured: 58% of an escrow across 20 intervals). Measuring escalation from `last_serviced_round` instead drops that to 22% — the first run of a burst clears the market, the rest pay base. The two features "multiply into something the creator cannot have modelled," so they were designed as one.

Why three call args, not sixteen. The obvious multi-argument *loop* is silently wrong (Puya hoists the inner transaction out of the loop, keeping only the last arg), so each argument count needs its own static branch, and program size grows super-linearly. Three (selector + two ABI args) is what fits on one program page alongside governance — and any arity is still reachable by packing arguments into a single ARC-4 struct.

Why the ASA fee is a bonus, not a denomination. A keeper's real costs are ALGO, and no contract can price your token without an oracle. Keeping a mandatory ALGO floor and adding the ASA *on top* lets the contract guarantee a keeper is never out of pocket without ever knowing what the ASA is worth. *"A capability is only a capability if the ALGO default is still complete on its own."*

Why staking (#15) and keeper-supplied data (#22) are closed, not deferred. Staking has nothing to slash (a keeper's only action is `execute`; a wrong execution is impossible; a failed one is already free). Keeper-supplied data inverts the one guarantee — the creator fixes *what* is called — and the useful version of it is an oracle network, a different product. *"A scheduled call is a heartbeat, not a courier."*

Appendix F — Glossary

Term	Meaning
Upkeep	A registered standing instruction: call this app with this data every N rounds, from this escrow. One box per upkeep.
Keeper	An off-chain process that calls `execute` on due upkeeps and collects fees.
Creator	The account that registered an upkeep; the only one who can cancel it.
Target app	The app an upkeep calls. Chosen and fixed by the creator.
Round	Algorand's block unit, ~2.66–2.8 s. Arcron's unit of time.
Escrow / balance	The ALGO an upkeep holds to pay for its executions.
Box deposit	The minimum-balance cost of an upkeep's box; refunded on cancel.
CATCH_UP / SKIP_AHEAD	Missed-run policy: replay every missed interval, or drop the backlog and keep phase.
fee_cap / escalation	An optional ceiling; a neglected upkeep's fee rises toward it. 0 = off.
Effective fee	The fee actually paid: base, or escalated when late and a ceiling is set.
Starved / stalled	Starved = escrow below one fee (creator's problem). Stalled = funded, due, unserviced (keeper's problem).
Frozen	Whether the creator has permanently given up the power to replace the programs.
µALGO	MicroALGO; 1 ALGO = 1,000,000 µALGO. The floor fee is 4,000 µALGO.
Pull pattern	Do accounting in the scheduled call; let counterparties collect in their own transactions.

Appendix G — The numbers, in one place

Deployment (TestNet, as of August 2026):

Keeper app — **use only this one**	`769891898` (alpha-3)
Pulse demo target	`769891902`
Console	`corvidlabs.xyz/arcron/console/`
CORVID asset (candidate, not wired in)	`3225439167`

Superseded — do not send funds to these: `769823086` (alpha-1, no update path), `769802474`, and `769772891` (both predate the 1.0 box shape). Current tooling refuses to decode them rather than misread, and the console quarantines any app id that is not `769891898`.

Limits and constants:

Minimum fee	4,000 µALGO (0.004 ALGO)
Console-suggested fee	10,000 µALGO (0.010 ALGO)
Maximum fee / cap	1,000,000,000 µALGO
Interval	10 – 1,000,000,000 rounds
Max app args	3 (including selector)
Max call data	1,024 bytes
ASA opt-in deposit	100,000 µALGO (permanent)
Box deposit (bare selector)	62,100 µALGO (refundable)
Keeper cost per execution	~3,000 µALGO (4,000 with an ASA bonus)
Keeper start floor	103,000 µALGO

Drift (rounds are not a clock; TestNet measured 2.66 s/round):

Cadence	Rounds	At 2.8 s	At 2.66 s	Drift/cycle
hourly	1,286	1.0 h	~0.95 h (57 min)	~3 min
daily	30,857	24.0 h	22.8 h	~1.2 h
weekly	216,000	168.0 h	159.7 h	~8.3 h

Runway (at the 4,000 µALGO floor):

Escrow	Executions	Hourly	Daily
0.1 ALGO	25	~1 day	~25 days
1 ALGO	250	~10 days	~8 months
100 ALGO	25,000	~2.8 years	~68 years

End of the guide. The source of truth is always the repository: `smart_contracts/keeper/contract.py` *for the contract,* `specs/keeper/` *for its ABI, and the live chain for anything this book states as a number.*